

Maestría en Inteligencia Artificial

Programación para Inteligencia Artificial

Primera clase

Introducción a la asignatura y a Python

El propósito de la asignatura

Software para trabajar con datos

Construiremos programas que validen información, transformen datos y conecten componentes de un sistema de IA.

Una solución que otra persona pueda usar

El resultado será código funcional, con pruebas e instrucciones para instalarlo y ejecutarlo. El entrenamiento de modelos corresponde a asignaturas posteriores.

El recorrido del curso

Unidad 1 · Python moderno

Fundamentos del lenguaje, contratos de datos y proyectos reproducibles.

Unidad 2 · Datos y representaciones

Arreglos, tablas, tensores y visualización de información.

Unidad 3 · Integración de componentes

Persistencia, consultas desde Python y flujos con estado.

Cierre · Proyecto integrador

Una solución local que combina lo aprendido y explica sus decisiones.

Unidad 1 · Programación eficiente en Python para IA

Fundamentos

Funciones, colecciones, decisiones, ciclos, excepciones y objetos.

Flujos y recursos

Iteradores, generadores y administradores de contexto.

Contratos de datos

Anotaciones de tipo y validación con Pydantic.

Reproducibilidad y calidad

Proyectos con uv, revisión con Ruff y pruebas con pytest.

Unidad 2 · Procesamiento y representación de datos para IA



NumPy

Arreglos, ejes, operaciones vectorizadas y broadcasting.

Formatos y tensores

CSV, Parquet e interoperabilidad con tensores de PyTorch.



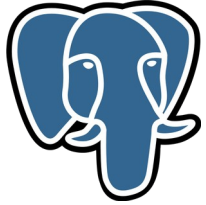
Pandas y datasets

Limpieza, combinación, faltantes y calidad de tablas.

Matplotlib

Gráficas para explorar datos y justificar hallazgos.

Unidad 3 · Integración de componentes para sistemas de IA



PostgreSQL

Tablas, consultas y persistencia relacional desde Python.

Interfaces

Responsabilidades y contratos entre componentes.
FastAPI como extensión opcional.



Neo4j

Entidades y relaciones como grafos, con consultas Cypher.

Orquestación

Estado, transiciones y manejo de fallos con
LangGraph o LangChain.

El proyecto integrador

Primer avance

Un problema acotado y datos con procedencia clara.

Procesamiento funcional y una primera escritura y consulta en la base de datos elegida.

Entrega final

Persistencia y un flujo de componentes con estado y fallos visibles.

Pruebas, documentación y una demostración que sustente las decisiones.

Evaluación del aprendizaje

25 % · Unidad 1

Ejercicios de Python moderno, validación y calidad.

30 % · Unidad 2

Proyecto de procesamiento y representación de datos.

20 % · Unidad 3

Proyecto de integración y orquestación.

25 % · Proyecto final

Integración, documentación y presentación.

Materiales y forma de trabajo

Repositorio del curso

[github.com/oscarnavmac/
programacion_ai](https://github.com/oscarnavmac/programacion_ai)

Los README organizan las unidades y contienen los enlaces de Google Colab.

Durante el taller

Predecir, ejecutar, explicar y modificar.

Notebook 1: fundamentos de Python.

Notebook 2: las 19 reglas del Zen.

Las soluciones están en Markdown para consultarlas después del intento.

Qué significa programar

Una tarea expresada con reglas

Identificar qué datos entran, qué operaciones se aplican y qué salida esperamos.

La computadora ejecuta las instrucciones que escribimos.

Ejemplo: resumir mediciones

Entrada: 20, 24 y 22 grados.

Operación: sumar y dividir entre tres.

Salida: 22 grados.

También debemos decidir qué hacer si faltan datos.

Qué es Python

Un lenguaje de propósito general

Su sintaxis permite expresar operaciones sobre números, texto y colecciones.

Un intérprete ejecuta el código. Python ofrece una biblioteca estándar y permite instalar bibliotecas adicionales.



Por qué se usa Python en IA

Código fácil de explorar

Las estructuras del lenguaje ayudan a expresar transformaciones y probar ideas.

Conexión con otros componentes

Python puede leer archivos, consultar bases de datos y coordinar servicios.

Bibliotecas especializadas

El ecosistema ofrece herramientas para cálculo numérico, datos y modelos.

Cálculo optimizado

Bibliotecas como NumPy delegan muchas operaciones a código compilado.

Python y sus bibliotecas

El lenguaje y la biblioteca estándar

Variables, funciones y ciclos pertenecen al lenguaje.

Herramientas como math y collections vienen con Python.

Hoy nos bastan esos recursos.

Bibliotecas que conoceremos

NumPy: arreglos numéricos.

Pandas: tablas y transformaciones.

PyTorch: tensores en este curso.

Matplotlib: visualización.

Una biblioteca agrega herramientas al lenguaje.

Un primer programa

Código Python

```
temperaturas = [20, 24, 22]
promedio = (
    sum(temperaturas) / len(temperaturas)
)
print(promedio)
```

Cómo leerlo

La lista reúne tres mediciones.

La segunda línea calcula el promedio y le asigna un nombre.

print muestra 22.0.

Una notebook combina texto y código

Celdas Markdown

Contienen explicaciones, preguntas y fragmentos de código para leer.

Un snippet dentro de Markdown no se ejecuta como una celda de código.

Celdas de código

Se ejecutan en una sesión de Python y pueden mostrar resultados.

La sesión conserva variables. El orden en que ejecutamos las celdas afecta al resultado.

La sesión de Python tiene memoria

Dos celdas, una misma sesión

Celda A

```
x = 10
```

Celda B

```
print(x + 1)
```

Después de ejecutar A, B muestra 11.

Un orden reproducible

Si la sesión es nueva y ejecutas B primero, falta el nombre x.

Reiniciar y ejecutar desde arriba permite comprobar que el resultado no depende de pasos ocultos.

Lo que practicaremos hoy

Representar datos

Números, cadenas, listas, diccionarios y otras colecciones.

Detectar problemas

Entradas inválidas, excepciones y comprobaciones con assert.

Transformar información

Funciones, decisiones, ciclos y comprensiones.

Conservar estado

Clases e instancias con un registro de predicciones como caso integrador.

Antes de abrir el taller

Dos preguntas para el grupo

¿Qué diferencia hay entre tener un valor cero y no tener un dato?

¿Qué debería hacer un programa cuando recibe texto donde esperaba un número?

Nuestra forma de aprender

Primero una predicción, luego la ejecución.

Un error ofrece información para revisar el programa.

Explicar una solución importa tanto como obtener una salida.

Primera práctica

Notebook 1

Curso acelerado de Python

[Abrir en Google Colab](#)

[Repositorio y guía de materiales](#)

